

1. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework

1-1. 이 논문이 진짜로 묻는 질문

MetaGPT의 문제의식은 단순하다.

기존 LLM 멀티에이전트 시스템은 간단한 대화형 작업은 되지만, 복잡한 작업에서는 LLM 들을 그냥 채팅으로 이어 붙이다 보니 hallucination이 연쇄적으로 누적되고, 논리 불일 치가 생긴다는 것이다. 그래서 저자들은 “에이전트 수를 늘리는 것”보다 인간 조직의 작업 절차(SOP)를 에이전트 협업에 이식하는 것이 더 중요하다고 본다.

즉, MetaGPT는

“LLM이 똑똑한가?”를 묻는 논문이 아니라,

“여러 LLM 역할이 섞여 일할 때, 어떤 프로토콜로 협업시키면 복잡한 작업을 덜 망치게 할 수 있는가?”를 묻는 논문이다.

1-2. 핵심 아이디어: “자유 대화”를 버리고 “업무 절차”를 넣는다

MetaGPT의 가장 중요한 주장은 이것이다.

복잡한 협업에서 자연어 잡담은 독이고, 구조화된 중간 산출물이 약이다.

논문 4-6쪽의 Figure 2와 Figure 3을 보면, MetaGPT는 소프트웨어 회사를 흉내 낸다. 사용자가 요구사항을 주면, 이를 여러 역할이 순차적으로 처리한다. Product Manager가 PRD를 만들고, Architect가 시스템 설계를 만들고, Project Manager가 태스크로 쪼개고, Engineer가 코드를 만들고, QA Engineer가 테스트를 만든다. 중요한 건 이들이 “서로 대화”하는 게 아니라, 정해진 문서 형식으로 핸드오프한다는 점이다.

논문은 이 구조를 assembly-line paradigm이라고 부른다. 즉, 각 역할은 자기 단계의 입력과 출력이 명확하고, 다음 단계는 그 산출물을 받아 작업한다. 이게 MetaGPT의 본질이다. 에이전트의 개성이 아니라 역할과 산출물의 표준화가 핵심이다.

1-3. 에이전트 구성: 누가 무엇을 하나

MetaGPT는 소프트웨어 개발 회사를 시뮬레이션하며 5개 역할을 둔다.

역할 하는 일

Product Manager 요구사항 분석, PRD 작성

Architect 시스템 설계, 파일 구조, 인터페이스, 데이터 구조 설계

Project Manager 태스크 분해 및 배분

Engineer 코드 작성

QA Engineer 테스트 작성, 품질 검증

각 agent는 이름, profile, goal, constraints를 가진다. 또 역할별로 쓸 수 있는 도구도 다르다. 예를 들어 Product Manager는 웹 검색 도구를, Engineer는 코드 실행 도구를 쓸 수 있다. 그리고 모든 agent는 환경, 즉 shared message pool을 관찰하면서 자신과 관련된 메시지가 오면 작업한다.

이 부분이 너희 프로젝트에 매우 중요하다.

MetaGPT는 “에이전트 = LLM 하나”가 아니라, 역할 + 책임 + 도구 + 입력/출력 계약의 조합으로 agent를 정의한다.

1-4. 왜 “구조화된 산출물”이 그렇게 중요한가

MetaGPT는 기존 멀티에이전트 프레임워크가 unconstrained natural language를 통신 인터페이스로 쓴다고 비판한다. 논문 6쪽에서 저자들은 telephone game 비유를 쓴다. 여러 번 전달되면 원래 정보가 왜곡된다는 것이다. 그래서 MetaGPT는 role마다 schema와 format을 정해 두고, 그 형식대로만 결과를 내게 한다.

예를 들어 Architect는 그냥 “설계 아이디어”를 말로 떠드는 게 아니라:

- system interface design
- sequence flow diagram
- file list
- data structures
- interface definitions

같은 문서/도표 형태를 만든다. 이것이 Engineer에게 전달된다. 논문은 ChatDev와 달리 MetaGPT는 대화가 아니라 문서와 다이어그램으로 소통한다고 분명히 말한다.

이게 왜 강력하냐면, 다음 agent 입장에서는

“앞 사람이 대충 뭐라고 말했더라?”가 아니라

“받아야 할 문서가 무엇인지, 그 문서 안에 있어야 할 항목이 무엇인지”가 명확해지기 때문이다.

즉, 협업 실패를 prompt engineering이 아니라 artifact engineering으로 줄인다는 생각이다.

1-5. Shared Message Pool과 Publish-Subscribe

MetaGPT의 두 번째 핵심은 메시지 풀이다.

각 agent가 서로 1:1로 계속 묻고 답하면 통신 그래프가 복잡해지고 비효율이 생긴다. 그래서 MetaGPT는 중앙 저장소 같은 global/shared message pool에 구조화된 메시지를 올린다. 에이전트들은 그 풀에서 필요한 정보만 가져간다.

그런데 모든 agent에게 모든 정보를 다 주면 정보 과부하가 생긴다. 그래서 저자들은 subscription mechanism을 둔다. 역할별 관심사에 따라 필요한 메시지만 구독하고, 필수 dependency가 다 들어왔을 때만 action을 activate한다. 예컨대 Architect는 Product Manager의 PRD는 꼭 봐야 하지만, QA Engineer의 문서는 당장 중요하지 않을 수 있다.

너희 프로젝트로 번역하면 이렇다.

- Strategy Agent는 DailyMarketPacket 없이는 실행되면 안 되고
- Risk Agent는 StrategyCard 없이는 실행되면 안 되고
- Backtest Agent는 OrderPlan 없이는 실행되면 안 된다

이 사고방식 자체가 MetaGPT에서 온다.

1-6. Executable Feedback: MetaGPT의 진짜 포인트

MetaGPT의 세 번째 핵심은 executable feedback이다.

저자들은 비실행형 코드 리뷰나 self-reflection만으로는 부족하다고 본다. 왜냐하면 LLM은 “맞는 것처럼 보이는 코드”를 써도 실제로는 실행이 안 되는 경우가 많기 때문이다

다. 그래서 Engineer가 코드를 생성한 뒤 직접 실행하고, 오류가 나면 과거 메시지, PRD, system design, existing code를 다시 참조해 수정하게 만든다. 테스트를 통과할 때까지, 혹은 최대 3번 재시도할 때까지 반복한다.

이건 단순히 “self-correction”이 아니다.

중요한 건 실행 결과가 다시 상위 reasoning loop에 들어간다는 점이다.

즉, 환경의 피드백이 협업 프로세스를 교정한다.

너희 금융 프로젝트에서 이걸 그대로 번역하면:

- StrategyAgent가 전략을 냄
- Risk/Backtest가 실제 시뮬레이션 결과를 냄
- 실패 카드(FailureCaseCard)를 발행
- Orchestrator가 제약을 강화해서 StrategyAgent를 재실행

이게 바로 금융판 executable feedback이다.

이 아이디어는 MetaGPT를 읽을 때 가장 먼저 가져와야 하는 부분이다.

1-7. 실험 결과가 의미하는 것

MetaGPT는 코드 생성 벤치마크 HumanEval과 MBPP에서 **Pass@1 85.9%, 87.7%**를 보고한다. 또 더 현실적인 SoftwareDev 벤치마크에서도 ChatDev보다 executability, human revision cost, productivity 측면에서 우수했다고 주장한다. 논문 8쪽 Table 10에 따르면, SoftwareDev에서 MetaGPT의 executability는 3.75, ChatDev는 2.25였고, human revision cost도 0.83 대 2.5로 낮았다.

더 중요한 건 ablation이다. 논문 9쪽 Table 3을 보면, Engineer 혼자 있을 때보다 Product/Architect/Project 역할이 추가될수록 executability와 revision cost가 좋아진다. 또 executable feedback를 넣으면 HumanEval/MBPP Pass@10이 각각 4.2%, 5.4% 올라가고, feasibility도 3.67에서 3.75로 개선된다. 즉, 성능 향상의 주된 원인을 저자들은 더 좋은 base model이 아니라 역할 분화 + 구조화 소통 + 실행 피드백에서 찾는다.

1-8. 이 논문을 한 문장으로 요약하면

MetaGPT는 “LLM agent를 더 똑똑하게 만드는 논문”이 아니라, “여러 agent가 망가지지 않게 일하도록 SOP와 structured artifact로 통제하는 논문”이다.

2. AAPM: Large Language Model Agent-based Asset Pricing Models

2-1. 이 논문이 진짜로 묻는 질문

AAPM은 TradExpert처럼 “바로 트레이딩을 잘하자”가 아니라, 더 금융경제학적인 질문을 던진다.

정성적(discretionary) 뉴스 분석에서 얻는 신호를, 기존의 수동 금융 팩터와 어떻게 결합하면 자산가격(asset pricing)을 더 잘 설명하고 예측할 수 있는가?

논문은 기존 asset pricing이 Fama-French류 manual factor에 크게 의존해 왔지만, 실제 discretionary investing은 뉴스, 조사, 리포트 같은 언어 정보에 크게 기대고 있다고

본다. 문제는 현재의 정량 팩터 모델이 그 정성 신호를 충분히 포착하지 못한다는 것이다. 그래서 AAPM은 LLM agent의 qualitative analysis와 manual factors를 함께 쓰는 hybrid asset pricing model을 제안한다.

즉, AAPM은

“LLM이 뉴스를 읽으면 주가가 오를지 내릴지 맞출 수 있나?”가 아니라,
“뉴스에서 추출한 정성적 상태 표현을 기존 팩터모형과 잘 결합하면 asset pricing 성능이 좋아지나?”를 묻는다.

2-2. 이 논문은 왜 “asset pricing”이지 “stock prediction”이 아닌가

여기서 중요한 개념 차이가 있다.

- stock movement prediction: 내일 오를지 내릴지, 혹은 수익률이 얼마일지 맞추는 문제
- asset pricing: 어떤 신호가 미래 초과수익(excess return)을 설명하고, anomaly portfolio의 pricing error를 줄이는지를 보는 문제

AAPM은 후자에 더 가깝다. 논문은 Sharpe ratio만 보지 않고, average $|\alpha|$, GRS test, anomaly portfolio pricing error 같은 금융경제학 지표를 함께 본다. 즉, 단순 방향 예측보다 설명력 있는 pricing signal에 더 관심이 있다.

이 차이를 이해해야 AAPM을 TradExpert와 혼동하지 않는다.

2-3. 전체 구조: Figure 1과 Figure 3가 말하는 것

AAPM의 2쪽 Figure 1과 4쪽 Figure 3이 논문의 핵심을 거의 다 보여준다. 파이프라인은 크게 두 부분이다.

1. LLM agent가 뉴스를 읽고 분석 보고서(report)를 생성
2. 그 보고서 임베딩을 manual factor와 결합해 pricing network에 넣음

더 자세히 보면:

- 최신 뉴스 x_t 가 들어온다.
- agent는 먼저 뉴스를 짧고 표준화된 형태로 refined news로 바꾼다.
- 동시에 macro/micro note를 입력으로 받는다.
- investment-relevant한 뉴스면 initial report를 만든다.
- 그 report를 바탕으로 memory vector base에 query를 날리고, top-K 관련 지식 조각을 가져온다.
- 이걸 반영해서 report를 N라운드 refinement한다.
- 최종 report는 note와 memory를 업데이트한다.
- 하루에 나온 모든 report의 embedding을 평균 내고, window smoothing을 해서 daily state를 만든다.
- 여기에 daily manual factors를 붙이고, asset-specific embedding을 더해 pricing network로 next-day excess return을 예측한다.

즉, AAPM의 구조는

뉴스 → reasoning report → embedding → factor fusion → return prediction 이다.

2-4. LLM agent 부분: iterative refinement + memory + notes

AAPM에서 가장 중요한 부분은 LLM agent 설계다.

첫 단계: refined news

raw news는 길고 형식이 제각각이라 바로 쓰기 어렵다. 그래서 agent가 먼저 핵심 정보를 요약한 x'_t를 만든다. 이 단계는 길이 제어와 입력 표준화 역할을 한다.

두 번째: relevance check

모든 뉴스가 투자적으로 의미 있지는 않다. agent는 뉴스가 investment-relevant한지 보고, 아니면 skip한다. 이 점은 너희 프로젝트에서 아주 중요하다. 뉴스가 많다고 다 쓰면 noise가 폭증한다.

세 번째: iterative refinement

초기 report R_t^0 를 만든 뒤 끝내지 않는다.

이전 report를 query로 삼아 external memory에서 관련 지식 top-K를 가져오고, 그걸 바탕으로 다음 report를 만든다. 이것을 N번 반복한다. 즉, 한 번 읽고 판단하는 게 아니라, “읽고 → 찾고 → 다시 해석하고 → 다시 찾는” 루프를 돈다.

네 번째: notes

AAPM은 단순 RAG만 쓰지 않는다.

macro note와 micro/market note를 유지한다. note는 현재 시장 분위기, 장기적으로 볼 아이디어, 최근 투자 관찰 같은 short-term/medium-term 문맥을 누적해 둔다. 그리고 새 report가 생성될 때마다 note도 업데이트된다. 즉, memory가 “지식 저장소”라면, note는 “현재 관찰과 시각의 누적 노트”에 가깝다.

이 구조는 실제 discretionary analyst가 하는 행동과 꽤 닮아 있다.

뉴스 하나를 읽고 끝내는 게 아니라:

- 지금 거시 국면이 어떤지 기억하고
- 이 회사에 대해 이전에 뭘 생각했는지 참고하고
- 추가 배경지식을 찾아본 뒤
- 최종 판단을 만든다

AAPM은 이 인간적 분석 흐름을 agent 설계로 옮기려는 논문이다.

2-5. 왜 knowledge cut-off 이후로 데이터셋을 시작했는가

AAPM은 매우 좋은 실험 설계 포인트를 갖는다.

논문은 note를 GPT-3.5-Turbo-1106의 knowledge cut-off date 이전에 초기화하고, 데이터셋은 그 cutoff 바로 다음부터 시작한다. 이유는 명확하다. LLM 사전학습 지식으로 생길 수 있는 정보 누수(leakage)를 줄이기 위해서다.

이건 금융 LLM 연구에서 매우 중요하다.

뉴스 timestamp를 잘 맞춰도, LLM이 이미 미래 사건을 pretraining에서 알고 있으면 평가가 오염될 수 있다. AAPM은 이 문제를 명시적으로 의식한다는 점에서 연구 태도가 좋다.

2-6. Hybrid pricing network: AAPM의 진짜 공학적 기여

LLM report만으로 예측하지 않는다는 점이 AAPM의 핵심이다.

논문 4-5쪽을 보면, 각 report를 embedding으로 바꾼 뒤 같은 날의 report embedding 평균을 daily embedding e_d 로 만들고, 이것을 lookback window LW 위에서 exponential decay kernel로 smoothing해서 s_d 를 만든다. 그리고 이 s_d 를 수동 금융경제 팩터 벡터 $v_{\{d,a\}}$ 와 concatenate한다. 이게 raw hybrid state다.

그 다음:

- downsampling layer
- asset-specific embedding lookup
- multi-layer prediction network

를 거쳐 next-day excess return을 예측한다. 또한 뉴스 데이터가 시작되기 전 과거 factor data로 네트워크를 pre-train하고, 그때는 placeholder embedding(예: “Null”)을 넣는다. 즉, 기존 factor model의 역사적 정보와 news-based language signal을 연속적으로 잇는 설계다.

이걸 한 문장으로 말하면:

AAPM의 진짜 기여는 “LLM이 분석 잘한다”가 아니라, “LLM report embedding과 manual factor가 noise가 아니라 synergy가 나도록 network를 설계했다”는 점이다.

2-7. 실험: 무엇을 이겼는가

AAPM은 NN, IPCA, CA, NF, CPZ 같은 최근 asset pricing baseline과 비교한다. 그리고 세 가지 포트폴리오 구성 방식—Tangency Portfolio, Equal-Weighted long-short decile, Value-Weighted long-short decile—에서 Sharpe ratio와 MDD를 본다. 저자들이 보고한 결과에 따르면, AAPM은 세 포트폴리오 모두에서 가장 높은 혹은 거의 가장 높은 Sharpe ratio를 기록했고, best baseline 대비 평균 9.6% Sharpe ratio 개선을 보였다. anomaly portfolio 기준 average $|\alpha|$ 도 10.8% 감소했다고 보고한다.

즉, AAPM은 “뉴스를 읽는 agent”를 넣어 성능이 좋아졌다고만 말하지 않는다. 포트폴리오 성능과 pricing error 두 축에서 다 개선됐다고 주장한다는 점이 중요하다.

2-8. Ablation: AAPM을 이해하려면 Table 3을 봐야 한다

AAPM의 핵심은 ablation이다.

논문 Table 3은 아래와 같은 계단식 구조를 보여준다.

- Naive: 뉴스만 보고 바로 report, daily embedding만 사용
- +RAG: 외부 memory 추가
- +Emb.: asset embedding 추가
- Memory: 둘 다 추가
- +Factors: manual factor 추가
- Hybrid: factor-history pretraining까지 한 hybrid network
- +Refine: iterative refinement 추가
- +Notes: macro/micro notes 추가
- Ours: refine + notes 모두 사용

결과를 보면, 단순 news embedding만 써도 기존 NF보다 조금 낫지만, RAG와 asset embedding이 붙으면서 좋아지고, factor와의 hybrid + pretraining에서 큰 점프가 생긴

다. 그 위에 refinement와 notes가 추가 이득을 준다. 최종적으로 SR은 Naive 2.82 → Ours 3.29, avg α 는 0.88 → 0.66으로 내려간다.

이 ablation이 말하는 건 명확하다.

1. LLM report alone은 불충분하다
2. memory/context가 중요하다
3. manual factors는 버릴 게 아니라 잘 결합해야 한다
4. refinement와 notes는 ‘있으면 좋은 옵션’이 아니라 실제로 의미 있는

증분을 만든다

2-9. N과 K: 얼마나 깊고 넓게 분석해야 하나

AAPM은 refinement 횟수 N과 retrieval 폭 K도 실험한다.

결과는 흥미롭다. N과 K가 늘면 대체로 좋아지지만, $K \times N \approx 15$ 근처에서 한계효용이 급격히 떨어진다. 또 $N=1$, $K=15$ 같은 극단은 오히려 성능이 떨어진다. 이유는 한 번 크게 긁어 오는 retrieval보다, 질문이 진화하면서 여러 번 refinement하는 과정이 더 정보적으로 풍부하기 때문이다.

이건 너희 프로젝트 설계에서도 중요하다.

“retrieval 많이 하면 좋겠지”가 아니라, 적당한 폭의 retrieval + 반복적 갱신이 더 낫다는 실험적 근거를 준다.

2-10. 한계

AAPM은 미국 시장과 영어 WSJ 뉴스에만 초점을 맞추고, knowledge cut-off 문제 때문에 데이터 기간도 2년으로 제한된다. 또 public information 중 뉴스 외의 광범위한 정보원, 그리고 multimodal input은 다루지 않는다. 저자들 스스로도 internet access, 더 다양한 정보원, specialized financial LLM, multimodal 정보가 향후 개선 포인트라고 본다.

2-11. 이 논문을 한 문장으로 요약하면

AAPM은 “뉴스를 읽는 LLM agent”를 만드는 논문이 아니라, “그 agent가 만든 정성 보고서를 manual factor와 결합해 asset pricing network로 쓰는 hybrid finance 논문”이다.

3. TradExpert: Revolutionizing Trading with Mixture of Expert LLMs

3-1. 이 논문이 진짜로 묻는 질문

TradExpert는 세 편 중 네 프로젝트 주제와 가장 직접적으로 닿아 있다. 문제의식은 이렇다.

금융 시장은 뉴스, 가격 시계열, 기술적 알파 요인, 펀더멘털 등 서로 다른 양식의 정보가 함께 움직이는 시스템인데, 이를 하나의 모델이 다 잘 소화하기 어렵다. 그렇다면 전문가 분업으로 나누는 게 낫지 않을까?

그래서 TradExpert는 Mixture of Experts 구조를 쓴다. 다만 일반적인 dense neural MoE가 아니라, financial data source별로 전문화된 여러 LLM을 두고, 그 결과를 General Expert가 종합하는 방식이다.

3-2. 입력 데이터가 네 종류라는 점이 출발점

TradExpert는 입력을 네 개로 나눈다.

- News: 뉴스 기사 텍스트
- Market Data: OHLCV 시계열
- Alpha Factors: 기술지표와 알고리즘적 팩터
- Fundamental Data: earnings call transcript + 재무지표

즉, TradExpert는 처음부터 “뉴스만 잘 읽는 모델”이나 “가격만 잘 보는 모델”이 아니라, 멀티소스 금융정보를 역할 분담으로 나눠 처리하는 구조로 설계되어 있다.

3-3. 전체 구조: Figure 2를 읽는 법

논문 3쪽 Figure 2가 핵심이다.

각 데이터 소스는 해당 Expert LLM으로 들어간다.

- News → News Analyst LLM → News Report
- Market Data → Market Analyst LLM → Market Report
- Alpha Factors → Alpha Expert LLM → Alpha Report
- Fundamental → Fundamental Analyst LLM → Fundamental Report

그 뒤 이 report들을 General Expert가 받아 최종 출력한다. 출력은 두 가지다.

1. Prediction mode: 한 종목이 다음 날 Rise/Fall 할지 예측
2. Ranking mode: 두 종목 중 누가 더 좋을지 비교하고, 그 비교를 반복해

Top-K를 고름

즉, TradExpert의 핵심은

“전문가별 report 생성 → general synthesis → decision” 이다.

3-4. 각 전문가가 실제로 뭘 하는가

News Analyst

뉴스 기사 텍스트를 받아 reasoning과 함께 Rise/Fall을 예측한다.

ground-truth reasoning은 GPT-4 API로 생성했다고 적혀 있다. 즉, supervision 데이터 안에 CoT 스타일 reasoning이 들어 있다.

Market Analyst

이 부분이 가장 기술적으로 흥미롭다.

OHLCV는 연속값 시계열이라 LLM token 구조와 안 맞는다. 그래서 TradExpert는 Time-LLM 계열 아이디어를 차용해 reprogramming을 쓴다. OHLCV를 patch embedding으로 자르고, LLM word embedding에서 유도한 text prototype과 multi-head cross-attention으로 정렬해 LLM이 받을 수 있는 형태의 embedding으로 바꾼다. 여기에 TSFresh 통계 설명까지 붙여 prompt에 넣는다.

Alpha Expert

108개 alpha factor의 수식을 GPT-4로 자연어 설명으로 바꾸고, 실제 factor 값은 LightGBM 기반 scoring model로 종합해서 Top-K 중요 alpha만 골라 prompt에 넣는다. 즉, Alpha Expert는 사실상 LLM + LightGBM 보조선별기 구조다. 이 점이 중요하다. TradExpert는 순수 LLM 시스템이 아니다.

Fundamental Analyst

earnings call transcript와 fundamental metric을 받아 다음 분기 기준으로 5단계 예측 (Strong Rise ~ Strong Fall)을 한다. 즉, 이 expert는 daily signal보다 더 긴 horizon을 본다.

General Expert

context length 한계 때문에 먼저 다른 expert들의 긴 대화/보고서를 summarize한다. 그 뒤:

- prediction mode에서는 한 종목 report를 받아 Rise/Fall 예측
- ranking mode에서는 두 종목 summary report를 비교해 누가 더 좋을지

판단

한다.

3-5. 왜 ranking mode가 중요하고, 왜 $O(N^2)$ 인가

TradExpert의 매우 독특한 포인트가 여기 있다. 일반적인 ranking model은 종목별 점수를 매기고 sort하면 되지만, TradExpert는 LLM comparator를 쓴다. 즉, “AAPL vs TSLA 중 내일 누가 더 나올까?” 같은 pairwise question을 던진다.

문제는 LLM 비교기가 비이행적(non-transitive) 이라는 점이다.

예를 들어:

- $A > B$
- $B > C$
- 그런데 $C > A$

같은 현상이 나올 수 있다. 그래서 QuickSort나 BubbleSort처럼 비교 횟수를 줄이는 알고리즘보다, 모든 쌍을 충분히 비교해 win count를 세는 relaxed comparison-based sorting을 택한다. 복잡도는 $O(N^2)$ 지만, 저자들은 실제로 이게 더 좋은 ranking을 준다고 보고한다. Table 6에서도 relaxed sort가 BubbleSort, QuickSort보다 RankIC/RankICIR에서 높다.

이건 TradExpert를 이해할 때 매우 중요하다.

이 논문은 단순 분류기 논문이 아니라, LLM을 ranking comparator로 쓰는 decision system 논문이기도 하다.

3-6. 데이터와 학습 방식

TradExpert는 2020-01-01부터 2023-12-31까지 4년 데이터를 쓴다.

뉴스는 여러 금융 뉴스 소스에서 수집한 524,995개 기사, 평균 596.4단어/기사이고,

Market Data는 S&P500 종목들의 OHLCV 기록 481,484건, Alpha는 108 technical indicators/factors, Fundamental은 Seeking Alpha 전사록과 재무지표다. 데이터 split은 시간순으로 train(2020-01-01~2022-06-30), validation(2022-07-01~2022-12-31), test(2023-01-01~2023-12-31)다.

또 모든 Expert LLM과 General Expert는 LLaMA-2-7B backbone + LoRA + supervised fine-tuning으로 학습한다. 부록에 따르면 learning rate $1e-4$, batch size 4, epochs 30, LoRA rank 8, alpha 16, dropout 0.1 등을 쓴다.

즉, TradExpert는 너희 현재 프로젝트 제약상 “아이디어를 그대로 구현”하기보다는, 분업 구조와 모듈 우선순위를 참고하는 게 더 현실적이다.

3-7. 실험 결과: 저자들이 주장하는 성과

주가 방향 예측

공개 benchmark와 proprietary S&P500 데이터에서, News+Market expert만 쓴 TradExpert-NMOI 여러 hybrid model과 LLM baseline 대비 대체로 우수했다고 보고한다. 특히 proprietary S&P500에서는 improvement가 더 컸다고 설명한다.

백테스트

DOW30을 대상으로 한 trading simulation에서, 저자들이 보고한 결과는 매우 강하다. TradExpert는 **AR 49.79%, AV 9.95%, SR 5.01, MD 6.56%**를 기록했고, A2C/PP0/EIIE/SARL/DeepTrader와 전통 모델들을 모두 앞섰다고 주장한다. Figure 6의 누적수익률 곡선도 가장 가파르게 오른다.

여기서 중요한 건 이 숫자를 “사실”로 받아들이기보다, 저자 실험 설정에서 그렇게 보고했다고 읽어야 한다는 점이다. 특히 proprietary dataset과 복잡한 LLM 구성 때문에 재현성은 따로 검증해야 한다.

3-8. Ablation: 어떤 전문가가 중요한가

TradExpert Table 4는 네 프로젝트에 매우 중요하다.

Market/News/Alpha/Fundamental 중 누구를 빼면 얼마나 망가지는지를 보여준다.

- full TradExpert: SR 5.01
- w/o Market: SR 1.88
- w/o News: SR 1.74
- w/o Alpha: SR 3.66
- w/o Fundamental: SR 4.15

즉:

1. Market Analyst와 News Analyst가 핵심
2. Alpha Expert는 분명 도움 되지만 1순위는 아님
3. Fundamental Analyst는 daily metric 영향은 가장 작지만 안정성엔 도움

이건 네 MVP 설계에 직접 영향을 준다.

스코프를 줄일 때는 Fundamental을 먼저 뒤로 미루고, Market과 News는 남겨야 한다는 근거가 바로 여기서 나온다.

3-9. Structured data reasoning도 검증한다

TradExpert는 “LLM이 구조화 데이터도 reasoning할 수 있나?”를 따로 본다.
TradExpert-MA를 alpha combination baseline과 비교했더니 RankIC 0.12 vs 0.07,
RankICIR 0.90 vs 0.65를 보고한다. 즉, 단순히 alpha score를 합치는 것보다, LLM이 정
형 데이터 위에서 reasoning을 수행한 결과가 더 낫다고 주장한다.

이 부분은 네 프로젝트에서 Market Analyst를 “숫자 설명기”로 돌지, 아니면 단순 규칙
기반 텍스트 변환기로 돌지를 고민할 때 참고할 가치가 있다.

3-10. 한계

TradExpert의 저자들은 한계를 솔직하게 적는다.
Nvidia A5000 GPU 기준 종목당 평균 4.7초가 걸린다. daily trading은 가능하지만,
high-frequency trading에는 부적합하다. 또한 ranking mode의 $O(N^2)$ 비교는 스케일이
커질수록 부담이 된다.

이 때문에 TradExpert를 실제 프로젝트에 넣을 때는 보통:

- 전 종목에 적용하지 않고
- shortlist에만 적용하며
- General Expert ranking도 소수 종목에만 쓰는

식으로 축소 구현하는 게 합리적이다.

3-11. 이 논문을 한 문장으로 요약하면

TradExpert는 “금융 데이터 소스별로 전문가 LLM을 분업시킨 뒤, General Expert가 이를
종합해 prediction 또는 ranking decision을 내리는 trading-oriented MoE 논문”이다.

4. 세 논문의 차이를 한 번에 정리

4-1. 무엇을 해결하려는 논문인가

논문	진짜 질문	핵심 산출물
MetaGPT	여러 agent를 어떻게 안정적으로 협업시킬까?	SOP, 구조화 문서, message pool, feedback
AAPM	뉴스 정성신호와 수동 팩터를 어떻게 결합할까?	analysis report + hybrid pricing state
TradExpert	다양한 금융 데이터 소스를 전문가 분업으로 어떻게 통합해 trading decision을 만들까?	expert reports + general decision

이 표의 핵심은 이거다.

MetaGPT는 협업 구조, AAPM은 정성-정량 융합, TradExpert는 전문가 분업형 금융 응용이
다.

4-2. 네 프로젝트에 가져와야 할 것

- MetaGPT에서 가져올 것

SOP, 구조화된 중간 산출물, shared registry/message pool, dependency-based handoff, executable feedback.

- AAPM에서 가져올 것

뉴스 refinement, memory/RAG, macro-micro notes, report embedding, factor fusion, leakage-aware evaluation mindset.

- TradExpert에서 가져올 것

Market/News 중심 전문가 분업, Alpha는 보조 expert, Fundamental은 후순위, General synthesizer, shortlist에만 LLM 적용하는 발상.

5. 네가 이 3편을 읽을 때 꼭 기억해야 할 오해 방지 포인트

5-1. MetaGPT는 금융 논문이 아니다

그래서 너희가 가져와야 하는 건 성능 숫자가 아니라 협업 문법이다. PRD 대신 StrategyCard, system design 대신 RiskCard/OrderPlan으로 치환하면 된다.

5-2. AAPM은 멀티에이전트 trading 논문이 아니다

실질적으로는 하나의 analytical agent + hybrid asset pricing network에 가깝다. 따라서 너희 프로젝트에선 memory/notes/fusion 구조를 빌려오면 된다.

5-3. TradExpert는 가장 직접적이지만 가장 무겁다

LoRA fine-tuning, 여러 expert LLM, pairwise ranking comparator까지 다 구현하면 학기 프로젝트에 과하다. 따라서 전문가 우선순위와 모듈 분리 로직을 배우는 것이 현실적이다.
